Tutorial

README · **Tutorial** · Reference

A **smoke test** checks that one part works at all: one small input, the right output. **Run them right after installing**, lowest part first, so you know every part works before the whole year. The first one that fails names the broken part.

Before you start: Set up. Gmail steps need an account. Each step: `python run.py smoke <step>`. Everything a smoke test makes is invented and lands in `Invoices/samples/<step>/`, apart from the real run in `Invoices/live/`.

Part 1: the senders

Step 1: one FCL and one LCL invoice, on Gmail

```
python run.py smoke freight --send
```

Proves: the system runs, end to end, into a real inbox.

Check by hand: in Gmail, one mail with two PDFs that open.

- FCL: containers × rate per container, about US$5,600 to 6,000 per 40'.
- LCL: rate × the greater of m^3 and tonnes, at least 1.
- Every line adds up to the total.

Step 2: one mail from each sender, on Gmail

```
python run.py smoke senders --send
```

Proves: the system delivers every flavour: every sender, every kind of document, text PDFs and scans, each error and its honest twin.

Check by hand: in Gmail, fifteen mails, one per sender: the thirteen regulars, the vendor with no order, and the new supplier.

- The body describes the sender, as the project imagines it: what it makes or moves, how and at what prices it sells, what it sends in a year, and how the simulation draws that to stay believable. It ends with a short note that the sender and its documents are invented.

- Factories: a goods invoice.
- MRK and CSM: FCL freight, and local charges in reais.
- DHI: LCL or air freight, and local charges in reais.
- SCA: a duty statement and a broker invoice.
- TVA: storage. TLV: truck. KNG: road haulage.
- Scans are a sender's habit, and the generator makes them: ARL sends a poor black-and-white scan laid up to 15 degrees askew, SCA a good straight one. Nothing on a scan can be selected.
- Each of the ten errors appears once, beside its honest twin; the answer key names both.
- Every line adds up; every value is in the reference's range.

Step 3: one sender forwards ten past PDFs, on Gmail

```
python run.py smoke forward SCA --send
```

Proves: the system runs loaded, with many documents in one mail.

Check by hand: in Gmail, one mail with exactly ten PDFs, under 25 MB.

- SCA, the broker, forwards the closed file of one past shipment: four goods invoices, the FCL freight, the local charges, the duty statement, its own fee, the storage and the truck. Only the broker holds a shipment's whole file, so only it can forward one.
- Every PDF is dated between 1 January and today.
- Every PDF opens and every line adds up.

Part 2: the reader

Step 4: read one mail from Gmail

```
python run.py smoke read-mail
```

Proves: the reader files both PDFs of step 1.

Check by hand: two rows in `Invoices/samples/freight/data/invoices.csv`; `first_seen` is when the mail arrived. Each reading step scores itself against the answer key in `score.json`: right, flagged for a person, wrong, missing. Wrong and missing must be empty.

Step 5: read one mail from each sender

```
python run.py smoke read-senders
```

Proves: the reader files every PDF of step 2.

Check by hand: one row per PDF; none missing.

- The scans go through OCR. An askew page is levelled first.
- A scan whose figures OCR misread, a 9 read as a 3, does not add up: its row says why and goes to a person. It is never filed as right.

Step 6: read the ten forwarded PDFs

```
python run.py smoke read-forward
```

Proves: the reader files the ten PDFs of step 3.

Check by hand: ten rows, each matching the answer key.

Step 7: cost one shipment

```
python run.py smoke cost
```

Proves: the documents of step 6 become the landed cost of each part.

Check by hand: in `Invoices/samples/cost/`, freight, local charges and truck split by volume, storage by value, duty by tariff code; the parts add up to the total the documents bill.

Part 3: the whole year, live

When every step above passes, start the dashboard, then the run. The dashboard runs in Docker inside WSL, from the WSL copy of the repo, and reaches the Python on Windows through its address:

```
cd monitoring && PIPELINE_HOST=$(ip route | awk '/default/ {print $3}') docker compose up -
```

```
python run.py request --send
```

```
python run.py backlog --send --metrics
```

```
python watch.py --metrics 9108
```

Proves: the whole period, 1 January to today, mailed the way each sender answers and read back as it lands.

A free Gmail account sends about 500 mails a day. If the send stops at the limit, it says how many are left; the same `backlog` command carries on later, or from a second account (how).

Check by hand: at `http://localhost:3000`, the real-run panels: replies sent, PDFs mailed, PDFs filed, left for a person. Then:

```
python run.py score
```

- Every document of `Invoices/live/ledger.json` is filed once; none is missing, none is wrong in silence. The rest are right or flagged.
- `python run.py cost` writes the cost report into `Invoices/live/cost/`.

Then run `python run.py live --send` each day: it brings only what was issued since the last run, one mail each. Every command, file and rate: the reference.